

Links:

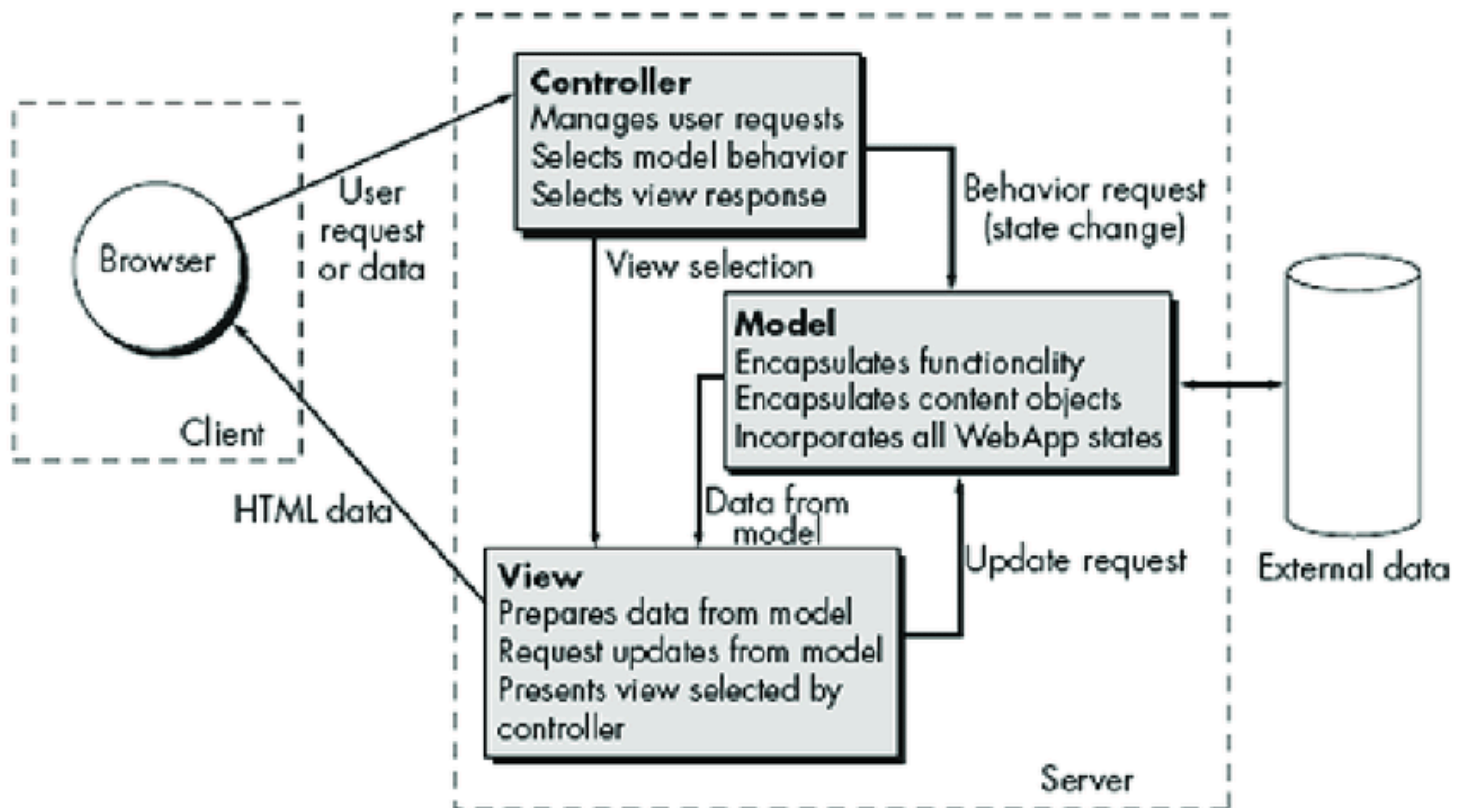
1. <https://www.nngroup.com/articles/ten-usability-heuristics/>

2.   [Builtin-filter](#)

3. Wireframe:

- Visual guide to represent structure of web page
- Information design
- Navigation design
- User interface design

MVC



 **origins:** [smalltalk-80](#) **Separation of responsibilities** >

It separated the application's data and logic (Model), its visual presentation (View), & the user's input handling (Controller). This

decoupling created reusable components and more manageable interfaces.

- **Model:** Application object

View

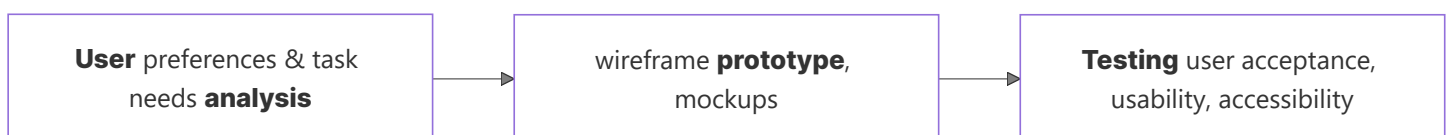
any (screen) **representation** useful to another entity
(HTML/JSON/XML/img)

- User interaction: keyboard, mouse, spoken voice (Hey Siri!) , custom buttons
 - Determined by:
 - hardware constraint
 - different target devices possible
 - user agent contextual info may not be in designer control but good to know
- User interface: screen, **audio, vibration HAPTIC, motor**
- Static: google.com, Partly dynamic: Wikipedia, Mostly dynamic: Amazon,

Controller

How user *interface* reacts to user *input*

- Prototyping - wireframes, mockups



Usability Heuristics

- Consistency

- Simple and minimal steps *user achieves goal with minimal effort*
- Simple language *easy to understand & use*
- Minimal and **aesthetic**ally pleasing
- **Visibility of System Status:** Always keep users informed about what is happening through timely & clear feedback.
- **Match Between System & the Real World:** Use language, concepts, & conventions familiar to users; avoid jargon & follow real-world logic.
- **User Control & Freedom:** Provide easy ways for users to undo or exit unwanted actions, giving them a sense of control.
- **Consistency & Standards:** Follow platform & industry conventions so users don't have to wonder if different words or actions mean the same thing.
- **Error Prevention:** Design to prevent errors before they occur, rather than just relying on good error messages.
- **Recognition Rather Than Recall:** Minimize memory load by making options, actions, & information visible & easily accessible.
- **Flexibility & Efficiency of Use:** Allow both novices & experts to use the system efficiently, offering shortcuts & customization for frequent users.
- **Aesthetic & Minimalist Design:** Keep interfaces simple & uncluttered, showing only relevant information & elements.
- **Help Users Recognize, Diagnose, & Recover from Errors:** Use plain language in error messages, clearly indicate problems, & suggest solutions.
- **Help & Documentation:** Provide easy-to-search, concise help & documentation focused on user tasks, available when needed.

```
import pyhtml as h
t = h.html(
    h.head(
        h.title('Test Page')
    ),
    h.body(
        h.h1('This is a title'),
        h.div('This is some text'),
        h.div(h.h2('inside title'),
            h.p('some text in a paragraph.'))
        )
    )
print(t.render())

def f_table(ctx):
    return (tr(
        td(cell) for cell in row
    ) for row in ctx['table'])
```

jinja template

Templates are like placeholders/variables for generating any output

- very limited programmability like *jinja macro & filters*

```

from jinja2 import Template

t = Template('$name is the $job of
$company')
s = t.substitute(name='Tim Cook',
job='CEO', company='Apple Inc.')
print(s)

t = Template('Hello {{ something }}') #{{
variable }}
print(t.render(something='World'))

t = Template('My favorite numbers: {% for n
in range(1,10) %}{{n}}' '% endfor %}')
print(t.render())

```

Accessibility

for vision, speech, touch, sensor-motor impairment W3C
(consortium)

perceivable

- Non-text/Multimedia control → **Alternative Text/Caption**
- Create content that can be presented in different ways without losing meaning  **assistive** tech
- easier to **see & hear** content (**speech**-oriented browsers)

operable

- All functionality available from **keyboard**, make it easier to use inputs other than 

- enough time to read & use content
- ❌ cause [seizures](#)/physical reactions
- Help users [navigate](#) & find content.

understandable

- text [readable](#) understandable
- Content appear & operate in [predictable](#) ways
- Help users avoid & ✅ [mistakes](#)

robust

- Max [compatibility](#) for future user tools.

aria-describedby

Inspect Developer Tools

```
console.table(array)
```



Jinja2 Cheatsheet

A quick reference guide for commonly used Jinja2 statements, filters, and patterns.

◆ Variable Printing

```
{{ variable }}  
{{ user.name }}  
{{ items[0] }}
```

👉 Outputs values from context.

◆ Template Inheritance (**extends** + **block**)

```
{% extends "base.html" %}  
{% block content %}  
    <h1>Hello</h1>  
{% endblock %}
```

NOTE: **super()** to call parent block.

```
{% block content %}  
    {{ super() }}    {# calls base.html content  
block #}  
    <p>Additional content</p>  
{% endblock %}
```

This will prepend the base content block(**super()'s output) to the block content.**

◆ Includes

```
{% include "navbar.html" %}
```

- **with context** → Passes current variables.
- **without context** → Isolates fragment.

Example:

```
{% include "sidebar.html" with context %}  
{% include "footer.html" without context %}
```

◆ Comments

```
{# This is a comment #}
```

◆ Control Structures

If / Else

```
{% if user %}  
    Hello, {{ user.name }}  
{% elif guest %}  
    Welcome guest  
{% else %}  
    Please login  
{% endif %}
```

For Loops

```
{% for item in items %}  
    <li>{{ loop.index }} - {{ item }}</li>  
{% endfor %}
```


Special loop vars: `loop.index`, `loop.first`, `loop.last`, `loop.revindex`.

Set Variables

```
{% set total = price * quantity %}
```

◆ With Statement (Scoped Variables)

```
{% with x = 5 %}  
    <p>{{ x }}</p>  
{% endwith %}
```

```
<p>{{ x }}</p>    {# undefined here #}
```

👉 Like `set`, but limited to block scope.

Example with Flask:

```
{% with messages =  
get_flashed_messages(with_categories=true)  
%}  
    {% if messages %}  
        {% for category, msg in messages %}  
            <div class="alert {{ category }}">{{  
msg }}</div>  
        {% endfor %}  
    {% endif %}  
{% endwith %}
```

◆ Macros (Reusable Functions)

```
{% macro render_input(name, type="text") %}  
    <input type="{{ type }}" name="{{ name  
}}">  
{% endmacro %}  
  
{{ render_input("email", "email") }}
```

◆ Importing Macros

```
{% import "forms.html" as forms %}
{{ forms.render_input("username") }}

{% from "forms.html" import render_input %}
{{ render_input("email") }}
```

◆ Call Blocks (Macros with Blocks)

```
{% macro panel(title) %}
  <div class="panel">
    <h3>{{ title }}</h3>
    <div class="panel-body">
      {{ caller() }}
    </div>
  </div>
{% endmacro %}

{% call panel("User Info") %}
  <p>Name: {{ user.name }}</p>
  <p>Email: {{ user.email }}</p>
{% endcall %}
```

👉 Use for wrapper components (cards, modals, etc.).

◆ Filters (Common)

```
{{ "hello world" | title }}      # Hello
World
{{ name | upper }}               #
UPPERCASE
{{ list | length }}              # Count
{{ value | default("N/A") }}    #
Fallback
{{ text | safe }}                # Don't
escape HTML
```

◆ Escaping

```
{% raw %}
    {{ this will not be evaluated }}
{% endraw %}
```

◆ Whitespace Control

```
{{- variable -}}
{% if cond -%} text {%- endif %}
```



Summary

- `extends` → Base layouts with `blocks`
- `include` → Reusable fragments (`with context` / `without context`)\
- `set` → Variables (global scope)
- `with` → Variables (block scope, e.g., flash messages)
- `macro` → Reusable function-like snippets
- `call` → Macros with blocks of HTML
- Filters → Modify data output